

# Tutorial Backend - Módulo Votos y Leaderboard

---

*Sistema de votaciones - 7.º Programación*

## Objetivo del grupo

Implementar el registro de votos, la comprobación básica de votos repetidos y el cálculo de rankings por categoría. Este módulo depende de Grupo y Categoría, por lo que debe respetar exactamente sus nombres e IDs.

## Estructura general del backend

```
backend/  
├── src/  
│   ├── config/  
│   │   └── db.js  
│   ├── controllers/  
│   ├── models/  
│   ├── routes/  
│   ├── services/  
│   └── app.js  
├── .env  
├── .env.example  
├── .gitignore  
├── package.json  
└── README.md
```

Cada grupo debe modificar únicamente los archivos correspondientes a su recurso, salvo que sea necesario coordinar un cambio común con el resto del equipo.

## Flujo obligatorio

Frontend → Route → Controller → Service → Model → MongoDB

- Route: define la URL y llama al controller.
- Controller: lee req.params, req.query o req.body; llama al service y responde con res.
- Service: contiene validaciones y lógica del recurso.
- Model: define el esquema de Mongoose y realiza las operaciones sobre MongoDB.
- El controller no consulta modelos directamente.
- El service no utiliza req ni res.

## Archivos asignados

- src/models/Voto.js
- src/services/votosService.js
- src/controllers/votosController.js
- src/routes/votosRoutes.js

## Convenciones que deben respetarse

- Todo el código y los nombres propios del proyecto estarán en español.
- Variables y funciones: camelCase.
- Modelos: singular y PascalCase, por ejemplo Grupo.js.
- Controllers: plural + Controller, por ejemplo gruposController.js.
- Routes: plural + Routes, por ejemplo gruposRoutes.js.
- Services: plural + Service, por ejemplo gruposService.js.
- URLs: minúsculas y sustantivos en plural.
- Usar const por defecto, let solo cuando el valor cambie.
- Comillas simples, punto y coma e indentación de dos espacios.
- Usar async/await para operaciones asíncronas.
- Exportar con module.exports.
- No guardar claves ni conexiones privadas dentro del código.

## Modelo de datos inicial

Los campos siguientes son una base de trabajo. Si durante el desarrollo se necesita agregar o cambiar un campo, debe acordarse antes con el equipo para no romper la integración.

- **grupoid:** Referencia al Grupo elegido.
- **categoriald:** Referencia a la Categoría votada.
- **identificadorDispositivo:** Identificador simple utilizado para reducir votos repetidos.
- **createdAt:** Fecha generada automáticamente mediante timestamps.

## Métodos que debe implementar el grupo

| Método                         | Responsabilidad                                       |
|--------------------------------|-------------------------------------------------------|
| registrarVoto                  | Validar y guardar un voto.                            |
| verificarVotoExistente         | Comprobar si un dispositivo ya votó en una categoría. |
| obtenerLeaderboard             | Obtener un resumen de resultados.                     |
| obtenerLeaderboardPorCategoría | Ordenar grupos por votos dentro de una categoría.     |

## Rutas que deben quedar disponibles

| HTTP | Ruta                                | Controller                     |
|------|-------------------------------------|--------------------------------|
| POST | /api/votos                          | registrarVoto                  |
| GET  | /api/votos/verificar                | verificarVotoExistente         |
| GET  | /api/votos/leaderboard              | obtenerLeaderboard             |
| GET  | /api/votos/leaderboard/:categoriald | obtenerLeaderboardPorCategoría |

## Orden recomendado de trabajo

| Paso | Archivo                  | Qué debe hacerse                                                                    |
|------|--------------------------|-------------------------------------------------------------------------------------|
| 1    | models/Voto.js           | Definir referencias a Grupo y Categoría e incluir el identificador del dispositivo. |
| 2    | services/votosService.js | Validar existencia de grupo/categoría, evitar repetidos                             |

|   |                                |                                                                                    |
|---|--------------------------------|------------------------------------------------------------------------------------|
|   |                                | y registrar el voto.                                                               |
| 3 | services/votosService.js       | Implementar el aggregate que calcula cantidad de votos por grupo y categoría.      |
| 4 | controllers/votosController.js | Recibir body/query/params y devolver respuestas HTTP.                              |
| 5 | routes/votosRoutes.js          | Crear las cuatro rutas acordadas.                                                  |
| 6 | Integración                    | Probar el módulo una vez que existan al menos un Grupo y una Categoría en la base. |

## Ejemplos base

Estos fragmentos sirven como referencia de estructura. Deben adaptarse a los campos definitivos y a las validaciones que acuerde el equipo.

### Modelo

```
const mongoose = require('mongoose');

const votoSchema = new mongoose.Schema({
  grupoId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Grupo',
    required: true
  },
  categoriaId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Categoría',
    required: true
  },
  identificadorDispositivo: {
    type: String,
    required: true
  }
}, { timestamps: true });

votoSchema.index(
  { categoriaId: 1, identificadorDispositivo: 1 },
  { unique: true }
);

module.exports = mongoose.model('Voto', votoSchema);
```

### Verificación en service

```
const Voto = require('../models/Voto');

async function verificarVotoExistente(categoriaId, identificadorDispositivo) {
  const voto = await Voto.exists({
    categoriaId,
```

```
    identificadorDispositivo
  });

  return Boolean(voto);
}
```

## Registro conceptual

```
async function registrarVoto(datosVoto) {
  const yaVoto = await verificarVotoExistente(
    datosVoto.categoriaId,
    datosVoto.identificadorDispositivo
  );

  if (yaVoto) {
    const error = new Error('Ya se registró un voto en esta categoría.');
```

error.status = 409;

throw error;

}

```
  return Voto.create(datosVoto);
}
```

## Leaderboard con aggregate

```
async function obtenerLeaderboardPorCategoria(categoriaId) {
  return Voto.aggregate([
    { $match: { categoriaId: new mongoose.Types.ObjectId(categoriaId) } },
    { $group: { _id: '$grupoId', votos: { $sum: 1 } } },
    { $sort: { votos: -1 } }
  ]);
}
```

## Pruebas mínimas antes de entregar

- Un voto válido se guarda correctamente.
- No se acepta grupoId inexistente.
- No se acepta categoriaId inexistente.
- El mismo identificador no puede votar dos veces en la misma categoría.
- El mismo identificador sí puede votar en categorías diferentes.
- El leaderboard ordena de mayor a menor.
- Los errores de voto repetido se responden con un código HTTP adecuado.
- No modificar los modelos Grupo o Categoria sin coordinarlo.

## Criterio de finalización

La parte asignada se considera lista cuando sus rutas responden correctamente, los errores básicos están controlados, los nombres coinciden con este documento y no se modificaron archivos pertenecientes a otros módulos sin coordinación.